

DSA

Module 17

Graphs

Author
Srinivas Dande





Java Learning Center

No.1 In Java Training & placement



17. Graphs

- ♦ Graph is a **Non-Linear Data Structure**
- ♦ In a graph, One element can follow multiple element and can be followed by multiple element.
- ♦ **Graph is a pair (V, E),**
 - **V** is a set of nodes, called vertices,
 - **E** is a collection of pairs of vertices, called edges.
 - **Vertices** and **Edges** are positions and store elements

17.1. Important Terminology

- 1) Directed Edge
- 2) Undirected Edge
- 3) Directed Graph:
- 4) Undirected Graph
- 5) Cyclic Graph
- 6) Acyclic Graph
- 7) Weighted Graph
- 8) Un-Weighted Graph
- 9) Connected Graph
- 10) DisConnected Graph
- 11) Walk
- 12) Path

1) Directed Edge:

- Ordered Pair of Vertices (v_1, v_2)
- We specify the Direction for Edge from One Vertex to Another Vertex

Ex:

- ✓ One-Way Road Traffic



2) Undirected Edge:

- Un-Ordered Pair of Vertices (v_1, v_2)
- No Direction for Edge from One Vertex to Another Vertex
- SO Bidirectional

Ex:

- ✓ Two-Way Road Traffic
- ✓ Railway Lines



3) Directed Graph:

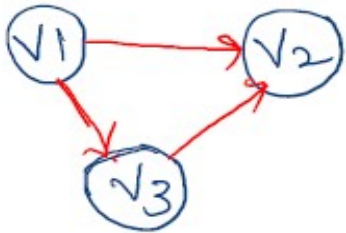
- Graph which contains **Directed Edges** is called as **Directed Graph**.
- All the edges are Directed

Ex:

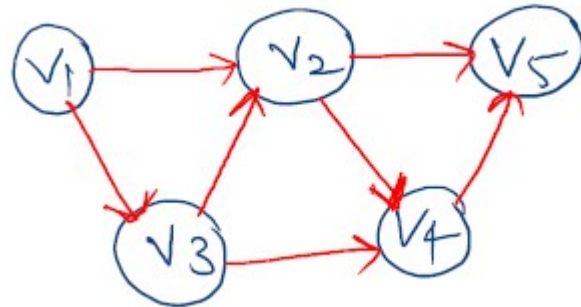
- ✓ Social Networks
- ✓ Transportation Networks
- ✓ Computer networks



Ex1:



Ex2:



- Every Vertex in the **Directed Graph** will have **Indegree** and **Outdegree**.
- **Indegree** is the number of incoming Edges to a Vertex,
- **Outdegree** is the number of outgoing Edges from a Vertex.

In Ex1:

Indegree(v1) = 0
Indegree(v2) = 2
Indegree(v3) = 1

Outdegree(v1) = 2
Outdegree(v2) = 0
Outdegree(v3) = 1

In Ex2:

Indegree(v1) = 0
Indegree(v2) = 2
Indegree(v3) = 1
Indegree(v4) = 2
Indegree(v5) = 2

Outdegree(v1) = 2
Outdegree(v2) = 2
Outdegree(v3) = 2
Outdegree(v4) = 1
Outdegree(v5) = 0

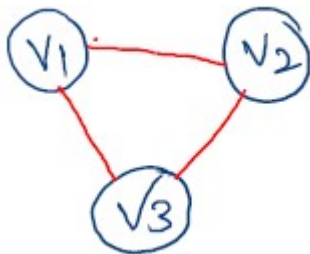
4) Undirected Graph:

- Graph which contains **Undirected Edges** is called as **Undirected Graph**.
- All the edges are Undirected

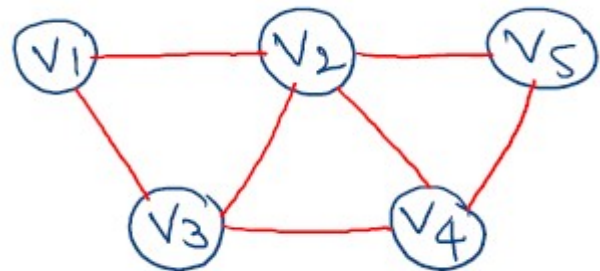
Ex:

- ✓ Flight Network
- ✓ Railway Lines

Ex1:



Ex2:



- Every Vertex in the **Undirected Graph** will have Degree
- **Degree** is the total number of Edges connected to it.

In Ex1:

Degree(v1) = 2

Degree(v2) = 2

Degree(v3) = 2

In Ex2:

Degree(v1) = 2

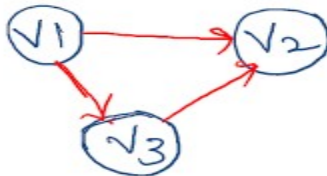
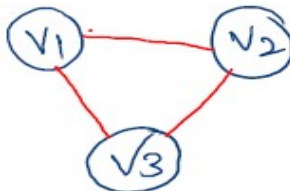
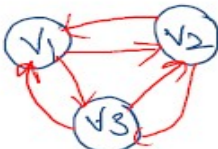
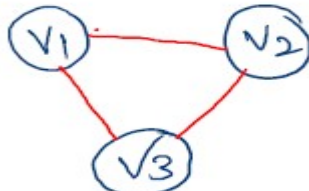
Degree(v2) = 4

Degree(v3) = 3

Degree(v4) = 3

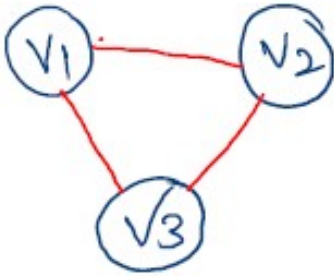
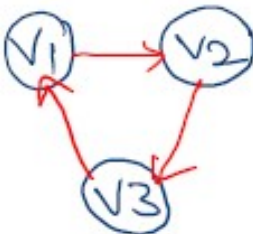
Degree(v5) = 2



Directed Graph	Undirected Graph									
1) Graph which contains Directed Edges is called as Directed Graph . ✓ Directed Edge is an Ordered Pair	1) Graph which contains Undirected Edges is called as Undirected Graph ✓ Undirected Edge is an Un-ordered Pair									
2) Ex: 	2) Ex: 									
3) Every Vertex in the Directed Graph will have Indegree and Outdegree <table><tr><td>Indegree(v1) = 0</td><td>Outdegree(v1) = 2</td></tr><tr><td>Indegree(v2) = 2</td><td>Outdegree(v2) = 0</td></tr><tr><td>Indegree(v3) = 1</td><td>Outdegree(v3) = 1</td></tr></table>	Indegree(v1) = 0	Outdegree(v1) = 2	Indegree(v2) = 2	Outdegree(v2) = 0	Indegree(v3) = 1	Outdegree(v3) = 1	3) Every Vertex in the Undirected Graph will have Degree <table><tr><td>Degree(v1) = 2</td></tr><tr><td>Degree(v2) = 2</td></tr><tr><td>Degree(v3) = 2</td></tr></table>	Degree(v1) = 2	Degree(v2) = 2	Degree(v3) = 2
Indegree(v1) = 0	Outdegree(v1) = 2									
Indegree(v2) = 2	Outdegree(v2) = 0									
Indegree(v3) = 1	Outdegree(v3) = 1									
Degree(v1) = 2										
Degree(v2) = 2										
Degree(v3) = 2										
4) Sum of Indegrees = E Sum of Outdegrees = E	4) Sum of Degrees = 2 * E									
5) Min. Number of Edges = 0	5) Min. Number of Edges = 0									
6) Max. Number of Edges = $V * (V-1)$ 	6) Max. Number of Edges = $V * (V-1) / 2$ 									

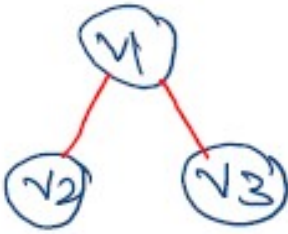
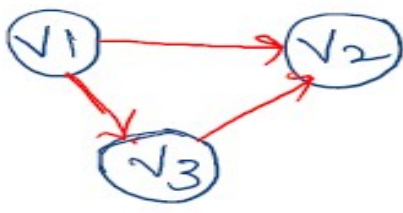
5) Cyclic Graph:

- **Cycle** is a Path where the First and Last Vertices are the same.
- If the Graph contains the Cycle, then that is called as **Cyclic Graph**
- Cyclic Graph can be Undirected Graph or Directed Graph

Ex1:  Undirected Cyclic Graph	Ex2:  Directed Cyclic Graph
--	---

6) ACyclic Graph:

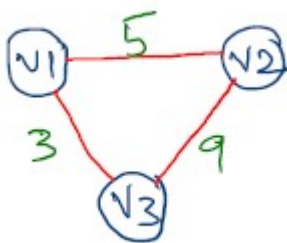
- If the Graph does not contain the Cycle, then that is called as **ACyclic Graph**
- ACyclic Graph can be Undirected Graph or Directed Graph

Ex1:  Undirected ACyclic Graph	Ex2:  Directed ACyclic Graph(DAG)
---	---

7) Weighed Graph:

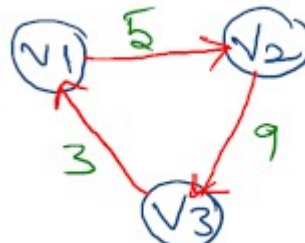
- We can assign weights to Edges.
- Weight is a Numeric Value assigned to Edge
- If the Edge contains the Weights, then that is called as **Weighted Graph**
- **Weighted** Graph can be Undirected Graph or Directed Graph

Ex1:



Undirected Weighted Graph

Ex2:

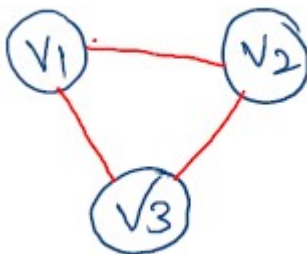


Directed Weighted Graph

8) Un-Weighed Graph:

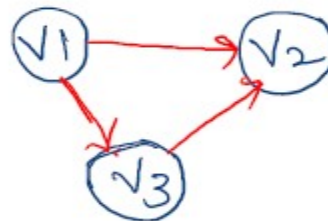
- If the Edge does not contain Weights, then that is called as **Un-Weighted Graph**
- **Un-Weighted** Graph can be Undirected Graph or Directed Graph

Ex1:



Undirected Un-Weighed Graph

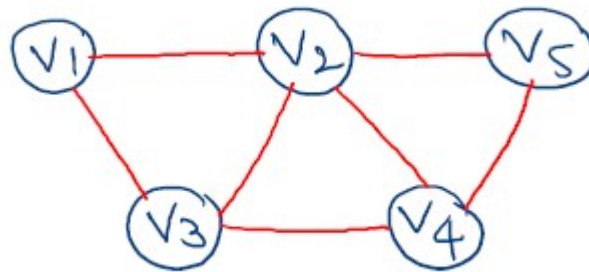
Ex2:



Directed Un-Weighed Graph

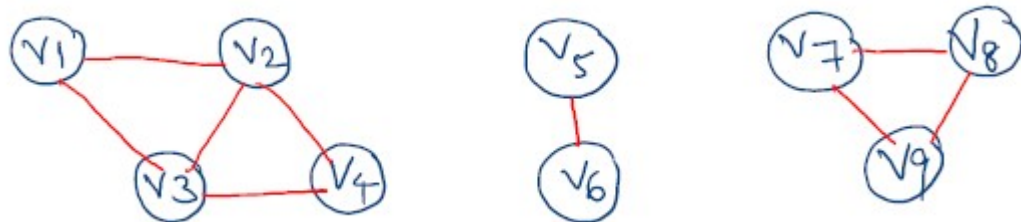
9) Connected Graph:

- Set of Vertices forms a Connected Component if any Vertex from the set of Vertices can reach any other Vertex by traversing edges.
- **The main point here is reachability.**
- If Graph contains Only One Connected Component, then that Graph is called as **Connected Graph**
- In Connected Graph, all Vertices are always reachable from each other



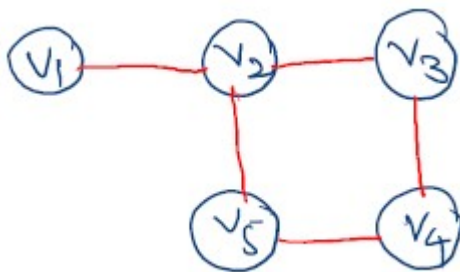
10) DisConnected Graph:

- If Graph contains Two or More Connected Components, then that Graph is called as **DisConnected Graph**
- In DisConnected Graph, all Vertices are Not reachable to each other



11) Walk:

- Walk is Sequence of Vertices Connected through Edges present in the Graph.
- Vertices can be Repeated in the Sequence.
- There are 2 Types of Walks
 - ✓ Open Walk
 - ✓ Closed Walk
- Open Walks will have different Starting and Ending Vertices.
- Closed Walks will have the same Starting and Ending Vertices.
- Trails are Open Walks with no Repeated Edges in the sequence



Ex:

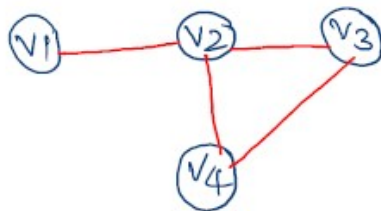
V1 – V2 – V3 -> Open Walk(Trail)

V1 – V2 – V3 – V2 -> Open Walk

V2 – V3 – V4 – V5 – V2 -> Closed Walk

12) Path:

- Path is Sequence of Non-Repeated Vertices Connected through Edges present in the Graph



Ex:

V1 – V2 – V3

V1 – V2 – V3 – V4

V1 – V2 – V4 – V3



17.2. Graph Representations

There are mainly 3 Popular Graph Representations

- 1) Adjacency Matrix
- 2) Adjacency List
- 3) Adjacency Set

1) Adjacency Matrix:

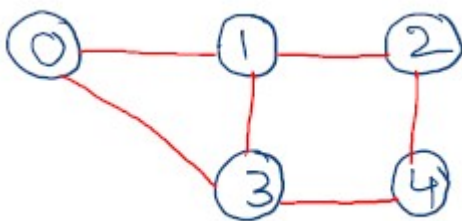
- Adjacency Matrix is the way of representing the Graph as a Matrix of 0's and 1's.
- Adjacency Matrix is represented as 2-D Array with V -Rows and V - Columns

Ex:

```
boolean[][] mat = new boolean[V][V];
```

- ✓ Size of Matrix is $V * V$ where V is the Number of Vertices
- ✓ Values of matrix are Boolean – **1 or 0**
- ✓ $Mat[i][j]$
 - Place **1** if there is an Edge from Vertex i to Vertex j
 - Place **0** if there is an No Edge from Vertex i to Vertex j

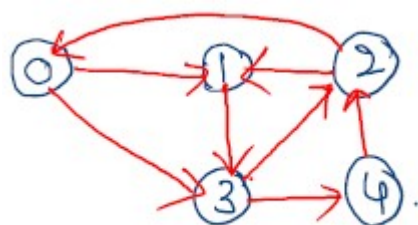
Undirected Graph



	0	1	2	3	4
0	0	1	0	1	0
1	1	0	1	1	0
2	0	1	0	0	1
3	1	1	0	0	1
4	0	0	1	1	0



Directed Graph



	0	1	2	3	4
0	0	1	0	1	0
1	0	0	0	1	0
2	1	1	0	0	0
3	0	0	1	0	1
4	0	0	1	0	0

2) Adjacency List:

- Adjacency List is another way of representing the Graph.
- Adjacency List stores only the required information about the Vertices which are connected.
- Adjacency List Saves the Space when compared with Adjacency Matrix because Adjacency Matrix stores information about the Vertices which are connected and also about the Vertices which are Not Connected.
- Adjacency List is represented as Array of Lists
- List can be represented as **ArrayList** or **LinkedList**

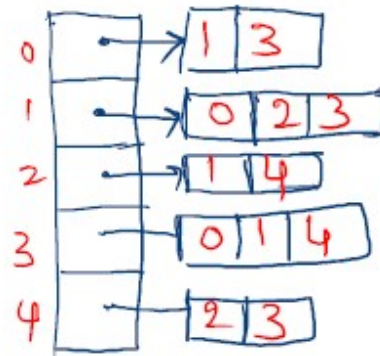
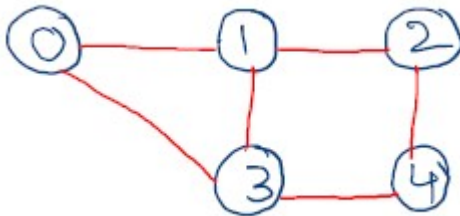
Ex:

```
ArrayList<ArrayList<Integer> list = new ArrayList<ArrayList<Integer>;
```

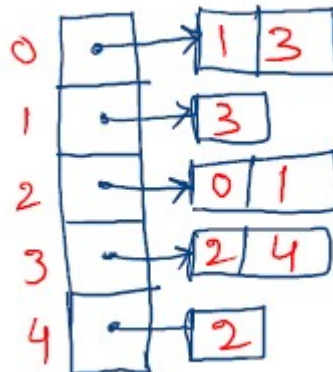
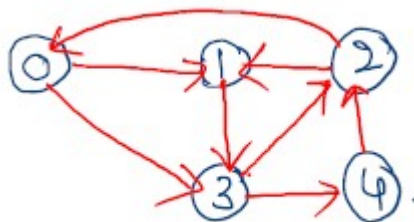
```
ArrayList<LinkedList<Integer> list = new ArrayList< LinkedList <Integer>;
```



Undirected Graph



Directed Graph



3) Adjacency Set:

- Adjacency Set Implementation is very much similar to Adjacency List but instead of using Linked lists, **Disjoint Sets** are used.

	Adjacency Matrix	Adjacency List
1) Space - For Directed Graph	$O(V * V)$	$O(V+E)$
2) Space - For Un-Directed Graph	$O(V * V)$	$O(V+2*E)$
3) Check v1 and v2 are Adjacent	$O(1)$	$O(V)$
4) Find All Adjacent Vertices of v1	$O(V)$	$O(\text{degree}(V))$
5) Find Degree of v1	$O(V)$	$O(1)$
6) Add an Edge	$O(1)$	$O(1)$
7) Remove an Edge	$O(1)$	$O(V)$



17.3. Adjacency List Implementation

17.3.1. Undirected Graph using Adjacency List

Lab1.java

```
package com.jlcindia.graphs.lab1;

import java.util.ArrayList;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyGraph {

    int vertexCount;
    ArrayList<ArrayList<Integer>> adjList;

    public MyGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<ArrayList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new ArrayList<Integer>());
        }
    }

    public void addEdge(int vertex1, int vertex2) { // 0,1
        adjList.get(vertex1).add(vertex2);
        adjList.get(vertex2).add(vertex1);
    }
}
```



```
public void printGraph() {
    for (int i = 0; i < vertexCount; i++) {
        System.out.print("Vertex : " + i + "=> \t");
        for (int j = 0; j < adjList.get(i).size(); j++) {
            System.out.print(adjList.get(i).get(j) + "\t");
        }
        System.out.println("");
    }
}

}

public class Lab1 {
    public static void main(String[] args) {

        MyGraph mygraph = new MyGraph(5);

        mygraph.addEdge(0, 1);
        mygraph.addEdge(0, 2);
        mygraph.addEdge(0, 3);
        mygraph.addEdge(0, 4);

        mygraph.addEdge(1, 2);
        mygraph.addEdge(1, 3);
        mygraph.addEdge(1, 4);

        mygraph.addEdge(2, 3);

        mygraph.addEdge(3, 4);

        mygraph.printGraph();
    }
}
```



17.3.2. Directed Graph using Adjacency List

Lab2.java

```
package com.jlcindia.graphs.lab2;

import java.util.ArrayList;
import java.util.LinkedList;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

class MyGraph {

    int vertexCount;
    ArrayList<LinkedList<Integer>> adjList;

    public MyGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<LinkedList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new LinkedList<Integer>());
        }
    }

    public void addEdge(int vertex1, int vertex2) {
        adjList.get(vertex1).add(vertex2);
    }
}
```



```
public void printGraph() {
    for (int i = 0; i < vertexCount; i++) {
        System.out.print("Vertex : " + i + "=> \t");
        for (int j = 0; j < adjList.get(i).size(); j++) {
            System.out.print(adjList.get(i).get(j) + "\t");
        }
        System.out.println("");
    }
}

}

}

public class Lab2 {
    public static void main(String[] args) {

        MyGraph mygraph = new MyGraph(5);

        mygraph.addEdge(0, 1);
        mygraph.addEdge(0, 2);

        mygraph.addEdge(1, 3);
        mygraph.addEdge(1, 4);

        mygraph.addEdge(2, 3);

        mygraph.addEdge(3, 4);

        mygraph.printGraph();
    }
}
```



17.4. BFS Traversal

17.4.1. BFS Traversal with Connected Graph

Lab3.java

```
package com.jlcindia.graphs.lab3;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Inputs : Undirected Connected Graph and Source Vertex
//This Graph may be Cyclic or Acyclic

class MyGraph {

    int vertexCount;
    ArrayList<ArrayList<Integer>> adjList;

    public MyGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<ArrayList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new ArrayList<Integer>());
        }
    }
}
```



```
public void addEdge(int vertex1, int vertex2) { // 0,1
    adjList.get(vertex1).add(vertex2);
    adjList.get(vertex2).add(vertex1);
}

public void bfsConnected(int sourceVertex) { // 0
    boolean visited[] = new boolean[vertexCount];
    bfs(sourceVertex,visited);
}

public void bfs(int sourceVertex,boolean visited[]) { // 0
    Queue<Integer> myque = new LinkedList<Integer>();

    myque.add(sourceVertex);
    visited[sourceVertex] = true;

    while (!myque.isEmpty()) {
        int myvertex = myque.poll();
        System.out.print(myvertex + "\t");

        for (int vertex : adjList.get(myvertex)) {
            if (!visited[vertex]) {
                myque.add(vertex);
                visited[vertex] = true;
            }
        }
    }
    System.out.println("");
}
}
```



```
public class Lab3 {  
    public static void main(String[] args) {  
  
        MyGraph mygraph = new MyGraph(5);  
  
        mygraph.addEdge(0, 1);  
        mygraph.addEdge(0, 2);  
  
        mygraph.addEdge(1, 3);  
        mygraph.addEdge(1, 4);  
  
        mygraph.addEdge(2, 3);  
  
        mygraph.addEdge(3, 4);  
  
        mygraph.bfsConnected(0);  
        System.out.println("-----");  
        mygraph.bfsConnected(2);  
        System.out.println("-----");  
        mygraph.bfsConnected(3);  
    }  
}
```



17.4.2. BFS Traversal with DisConnected Graph

Lab4.java

```
package com.jlcindia.graphs.lab4;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Inputs : Undirected DisConnected Graph
//This Graph may be Cyclic or Acyclic

class MyGraph {

    int vertexCount;
    ArrayList<ArrayList<Integer>> adjList;

    public MyGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<ArrayList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new ArrayList<Integer>());
        }
    }

    public void addEdge(int vertex1, int vertex2) { // 0,1
        adjList.get(vertex1).add(vertex2);
        adjList.get(vertex2).add(vertex1);
    }
}
```



```
public void bfsDisconnected() { // 0
    boolean visited[] = new boolean[vertexCount];

    for (int i = 0; i < vertexCount; i++) {
        if (!visited[i]) {
            bfs(i, visited);
        }
    }
}

public void bfs(int sourceVertex, boolean visited[]) { // 0
    Queue<Integer> myque = new LinkedList<Integer>();

    myque.add(sourceVertex);
    visited[sourceVertex] = true;

    while (!myque.isEmpty()) {
        int myvertex = myque.poll();
        System.out.print(myvertex + "\t");

        for (int vertex : adjList.get(myvertex)) {
            if (!visited[vertex]) {
                myque.add(vertex);
                visited[vertex] = true;
            }
        }
    }
}
```



```
public class Lab4 {  
    public static void main(String[] args) {  
  
        MyGraph mygraph = new MyGraph(8);  
  
        mygraph.addEdge(0, 1);  
        mygraph.addEdge(0, 2);  
        mygraph.addEdge(1, 2);  
  
        mygraph.addEdge(3, 4);  
  
        mygraph.addEdge(5, 6);  
        mygraph.addEdge(5, 7);  
  
        mygraph.bfsDisconnected();  
  
    }  
}
```



17.4.3. Count Connected Components in Graph

Lab5.java

```
package com.jlcindia.graphs.lab5;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Inputs : Undirected DisConnected Graph
//This Graph may be Cyclic or Acyclic
// Count the Islands in DisConnected Graph

class MyGraph {

    int vertexCount;
    ArrayList<ArrayList<Integer>> adjList;

    public MyGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<ArrayList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new ArrayList<Integer>());
        }
    }

    public void addEdge(int vertex1, int vertex2) { // 0,1
        adjList.get(vertex1).add(vertex2);
        adjList.get(vertex2).add(vertex1);
    }
}
```



```
public int countIslands() { // 0
    boolean visited[] = new boolean[vertexCount];
    int count = 0;
    for (int i = 0; i < vertexCount; i++) {
        if (!visited[i]) {
            bfs(i, visited); //
            count++;
        }
    }
    return count;
}

public void bfs(int sourceVertex, boolean visited[]) { // 0
    Queue<Integer> myque = new LinkedList<Integer>();

    myque.add(sourceVertex);
    visited[sourceVertex] = true;

    while (!myque.isEmpty()) {
        int myvertex = myque.poll();

        for (int vertex : adjList.get(myvertex)) {
            if (!visited[vertex]) {
                myque.add(vertex);
                visited[vertex] = true;
            }
        }
    }
}
```



```
public class Lab5 {  
    public static void main(String[] args) {  
  
        MyGraph mygraph = new MyGraph(10);  
  
        mygraph.addEdge(0, 1);  
        mygraph.addEdge(0, 2);  
        mygraph.addEdge(1, 2);  
  
        mygraph.addEdge(3, 4);  
  
        mygraph.addEdge(5, 6);  
        mygraph.addEdge(5, 7);  
  
        mygraph.addEdge(8,9);  
  
        int count = mygraph.countIslands();  
        System.out.println(count);  
  
    }  
}
```



17.5. DFS Traversal

17.5.1. DFS Traversal with Connected Graph

Lab6.java

```
package com.jlcindia.graphs.lab6;

import java.util.ArrayList;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Inputs : Undirected Connected Graph and Source Vertex
//This Graph may be Cyclic or Acyclic
class MyGraph {
    int vertexCount;
    ArrayList<ArrayList<Integer>> adjList;

    public MyGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<ArrayList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new ArrayList<Integer>());
        }
    }
    public void addEdge(int vertex1, int vertex2) { // 0,1
        adjList.get(vertex1).add(vertex2);
        adjList.get(vertex2).add(vertex1);
    }
    public void dfsConnected(int sourceVertex) { // 0
        boolean visited[] = new boolean[vertexCount];
        dfs(sourceVertex,visited);
    }
}
```



```
public void dfs(int myvertex,boolean visited[]) { // 0

    visited[myvertex] = true;
    System.out.print(myvertex + "\t");

    for (int vertex : adjList.get(myvertex)) { //1,2
        if (!visited[vertex]) {
            dfs(vertex,visited);
        }
    }
}

}

public class Lab6 {
    public static void main(String[] args) {

        MyGraph mygraph = new MyGraph(6);

        mygraph.addEdge(0, 1);
        mygraph.addEdge(0, 2);
        mygraph.addEdge(1, 3);
        mygraph.addEdge(1, 5);
        mygraph.addEdge(2, 3);
        mygraph.addEdge(2, 4);
        mygraph.addEdge(3, 4);
        mygraph.addEdge(4,5);

        mygraph.dfsConnected(0);
        System.out.println("-----");
        mygraph.dfsConnected(2);
        System.out.println("-----");
        mygraph.dfsConnected(3);
    }
}
```



17.5.2. DFS Traversal with DisConnected Graph

Lab7.java

```
package com.jlcindia.graphs.lab7;

import java.util.ArrayList;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Inputs : Undirected DisConnected Graph
//This Graph may be Cyclic or Acyclic
class MyGraph {
    int vertexCount;
    ArrayList<ArrayList<Integer>> adjList;

    public MyGraph(int vertexCount) {
        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<ArrayList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new ArrayList<Integer>());
        }
    }
    public void addEdge(int vertex1, int vertex2) { // 0,1
        adjList.get(vertex1).add(vertex2);
        adjList.get(vertex2).add(vertex1);
    }
    public void dfsDisconnected() { // 0
        boolean visited[] = new boolean[vertexCount];
        for (int i = 0; i < vertexCount; i++) {
            if (!visited[i]) {
                dfs(i, visited);
            }
        }
    }
}
```



```
public void dfs(int myvertex,boolean visited[]) { // 0

    visited[myvertex] = true;
    System.out.print(myvertex + "\t");

    for (int vertex : adjList.get(myvertex)) { //1,2
        if (!visited[vertex]) {
            dfs(vertex,visited);
        }
    }
}

}

public class Lab7 {
    public static void main(String[] args) {

        MyGraph mygraph = new MyGraph(8);

        mygraph.addEdge(0, 1);
        mygraph.addEdge(0, 2);
        mygraph.addEdge(1, 2);

        mygraph.addEdge(3, 4);

        mygraph.addEdge(5, 6);
        mygraph.addEdge(5, 7);

        mygraph.dfsDisconnected();

    }
}
```



17.5.3. Count Connected Components in Graph

Lab8.java

```
package com.jlcindia.graphs.lab8;

import java.util.ArrayList;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

// Inputs : Undirected DisConnected Graph
//This Graph may be Cyclic or Acyclic
//Count the Islands in DisConnected Graph

class MyGraph {

    int vertexCount;
    ArrayList<ArrayList<Integer>> adjList;

    public MyGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<ArrayList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new ArrayList<Integer>());
        }
    }

    public void addEdge(int vertex1, int vertex2) { // 0,1
        adjList.get(vertex1).add(vertex2);
        adjList.get(vertex2).add(vertex1);
    }
}
```



```
public int countIslands() { // 0
    boolean visited[] = new boolean[vertexCount];

    int count=0;
    for (int i = 0; i < vertexCount; i++) {
        if (!visited[i]) {
            dfs(i, visited);
            count++;
        }
    }
    return count;
}

public void dfs(int myvertex, boolean visited[]) { // 0

    visited[myvertex] = true;
    for (int vertex : adjList.get(myvertex)) { // 1,2
        if (!visited[vertex]) {
            dfs(vertex, visited);
        }
    }
}

}

public class Lab8 {
    public static void main(String[] args) {

        MyGraph mygraph = new MyGraph(8);

        mygraph.addEdge(0, 1);
        mygraph.addEdge(0, 2);
        mygraph.addEdge(1, 2);
        mygraph.addEdge(3, 4);
        mygraph.addEdge(5, 6);
        mygraph.addEdge(5, 7);
    }
}
```



```
int count = mygraph.countIslands();
System.out.println(count);

MyGraph mygraph1 = new MyGraph(10);

mygraph1.addEdge(0, 1);
mygraph1.addEdge(0, 2);
mygraph1.addEdge(1, 2);

mygraph1.addEdge(3, 4);

mygraph1.addEdge(5, 6);
mygraph1.addEdge(5, 7);

mygraph1.addEdge(8,9);

count = mygraph1.countIslands();
System.out.println(count);
    }
}
```